Spatial Analysis and Modeling

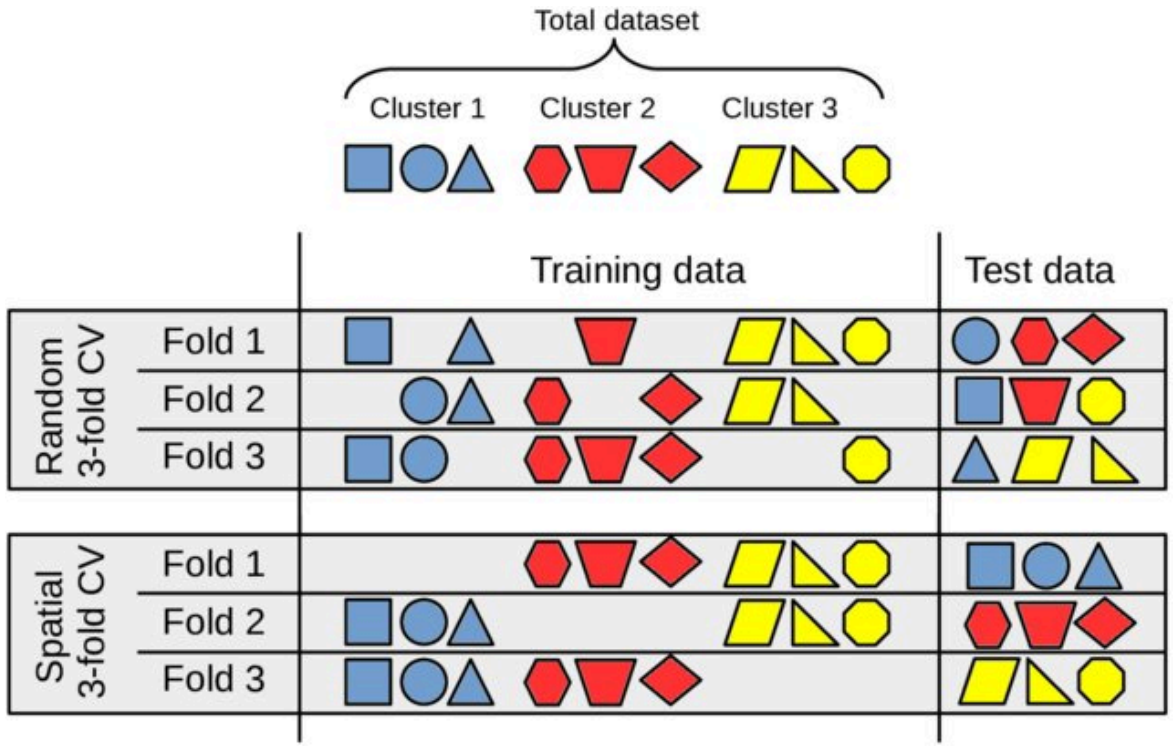
Spatial Cross-Validation

Why Standard CV Fails in Spatial Data

The Core Problem: Information Leakage

When your data has spatial autocorrelation, random K-fold CV creates a fundamental problem:

- **Train and test folds contain nearby locations** that share information
- This makes predictions appear **easier than they really are**
- Your model gets "hints" about test data through spatially correlated neighbors
    - Violates the **independence assumption** that CV relies on



Block Spatial Cross-Validation

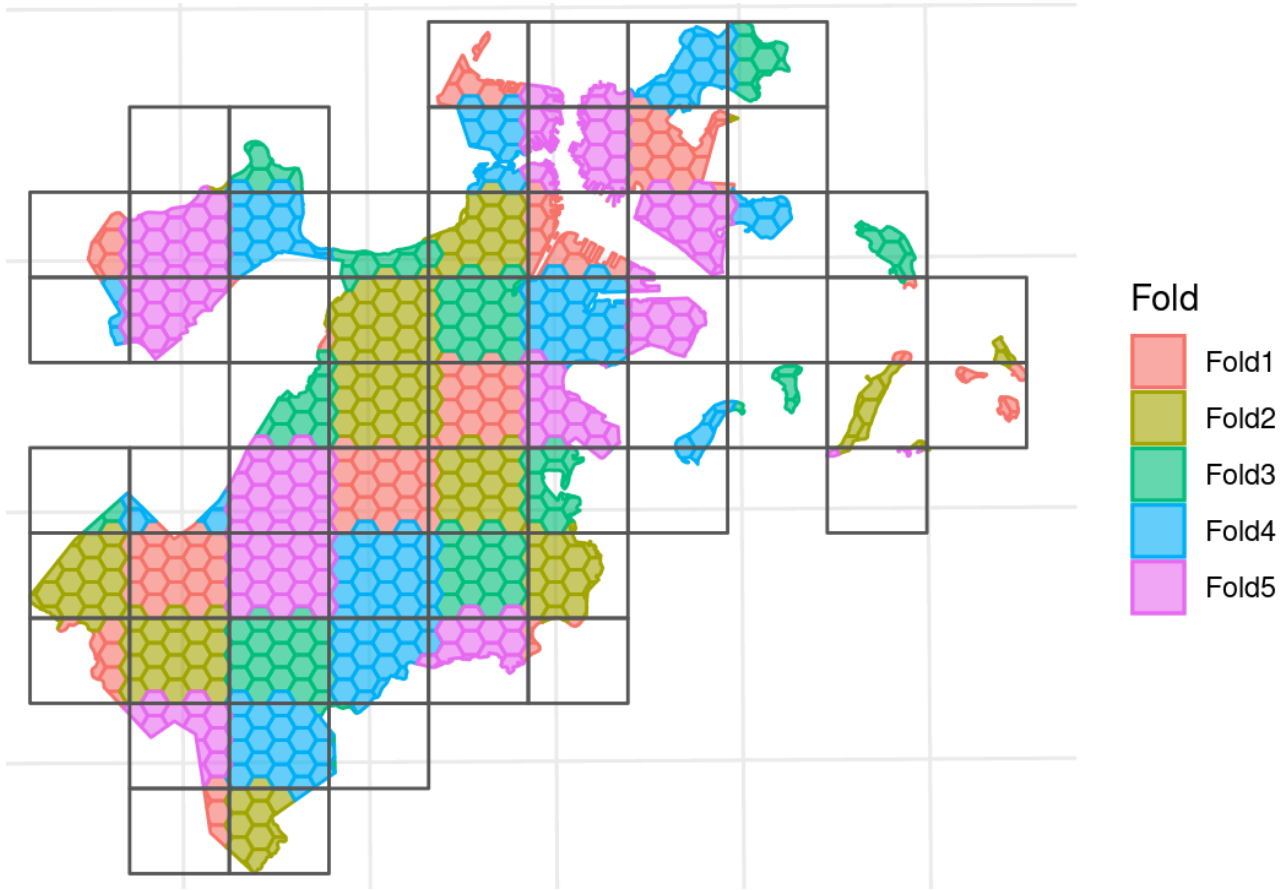
Concept: Partition space into contiguous blocks

Advantages:

- Preserves spatial structure
- Realistic evaluation scenario
- Simple to implement

Disadvantages:

- Unequal block sizes possible
- Edge effects between blocks



Clustering-based Spatial Cross-Validation

Rather than carving space into a regular grid, we let the data define the folds: cluster the observations, then hold out one whole cluster at a time. Feeding covariates into the clustering lets folds follow environmental gradients, not just geographic distance.

k-means recap (used here; taught fully in deck 06): k-means partitions points into k groups, repeatedly assigning each point to the nearest cluster centre and recomputing the centres to minimise within-cluster squared distance.

Concept:

- Instead of regular spatial blocks, you can use **clustering** to define folds.
- Clusters can be formed using spatial coordinates or additional covariates (e.g., elevation, land cover, climate).
- Each cluster is treated as a test fold, with the remaining clusters as training data.

How it works:

- Clustering groups nearby or similar observations, potentially capturing complex spatial or environmental patterns.
- Covariate-based clustering allows folds to reflect ecological or environmental similarity, not just geographic proximity.

Clustering-based CV — trade-offs

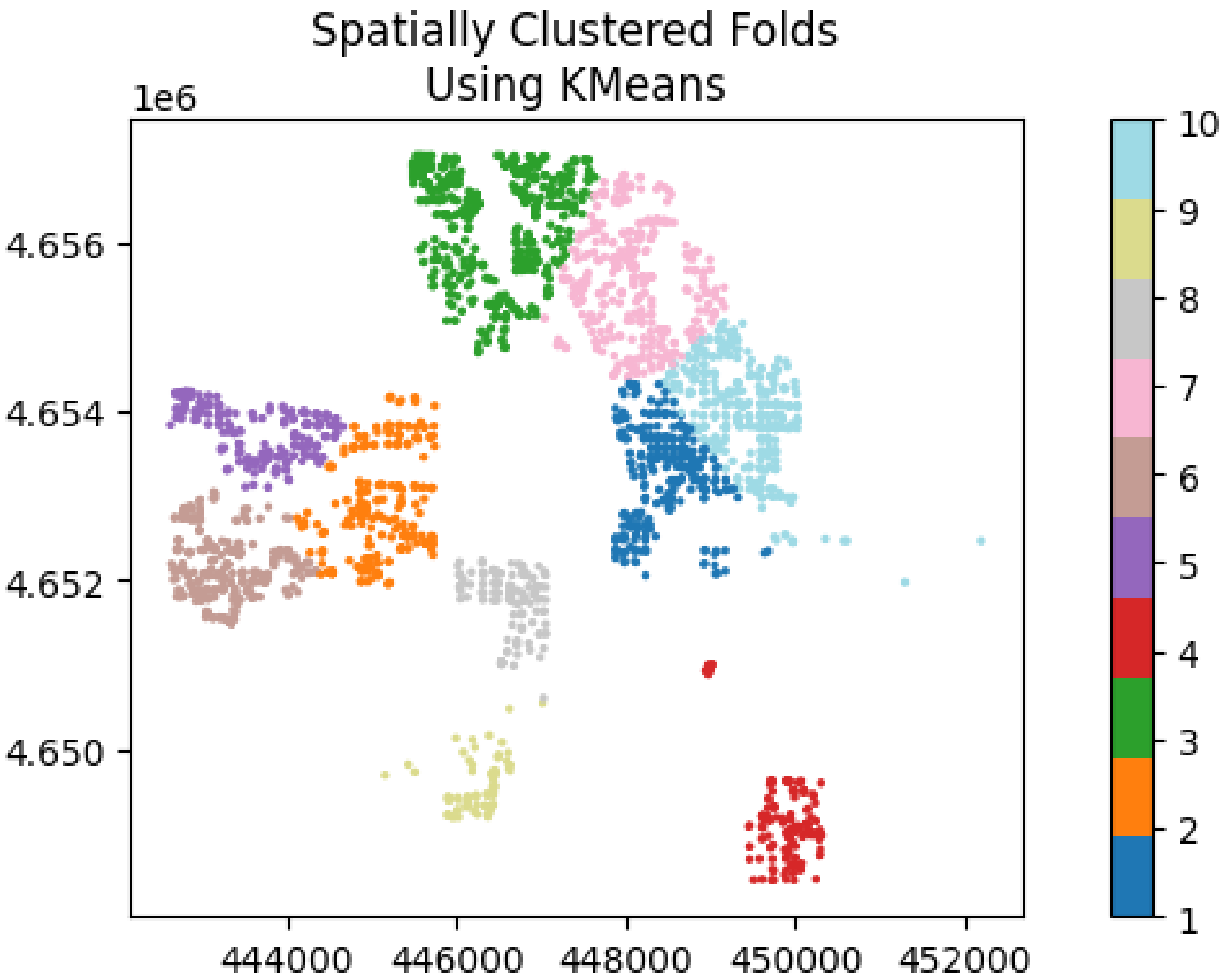
Clusters buy ecological realism but cost you control: fold sizes wander and groups need not be contiguous, so weigh these before trusting the scores.

Advantages:

- Can create more ecologically meaningful folds, especially when covariates are used.
- Flexible: clusters can be spatially contiguous or disjoint, depending on the algorithm and input features.
- Useful for testing model transferability across environmental gradients.

Disadvantages:

- Fold sizes may be uneven, especially with natural clusters.
- Clusters may not be spatially contiguous, which can complicate interpretation.
- Choice of clustering algorithm and covariates can strongly affect results.
- May not fully eliminate spatial autocorrelation between folds if clusters are too small or too close.


Spatially Clustered Folds
Using KMeans
1e6
4.656
4.654
4.652
4.650
444000
446000
448000
450000
452000
10
9
8
7
6
5
4
3
2
1

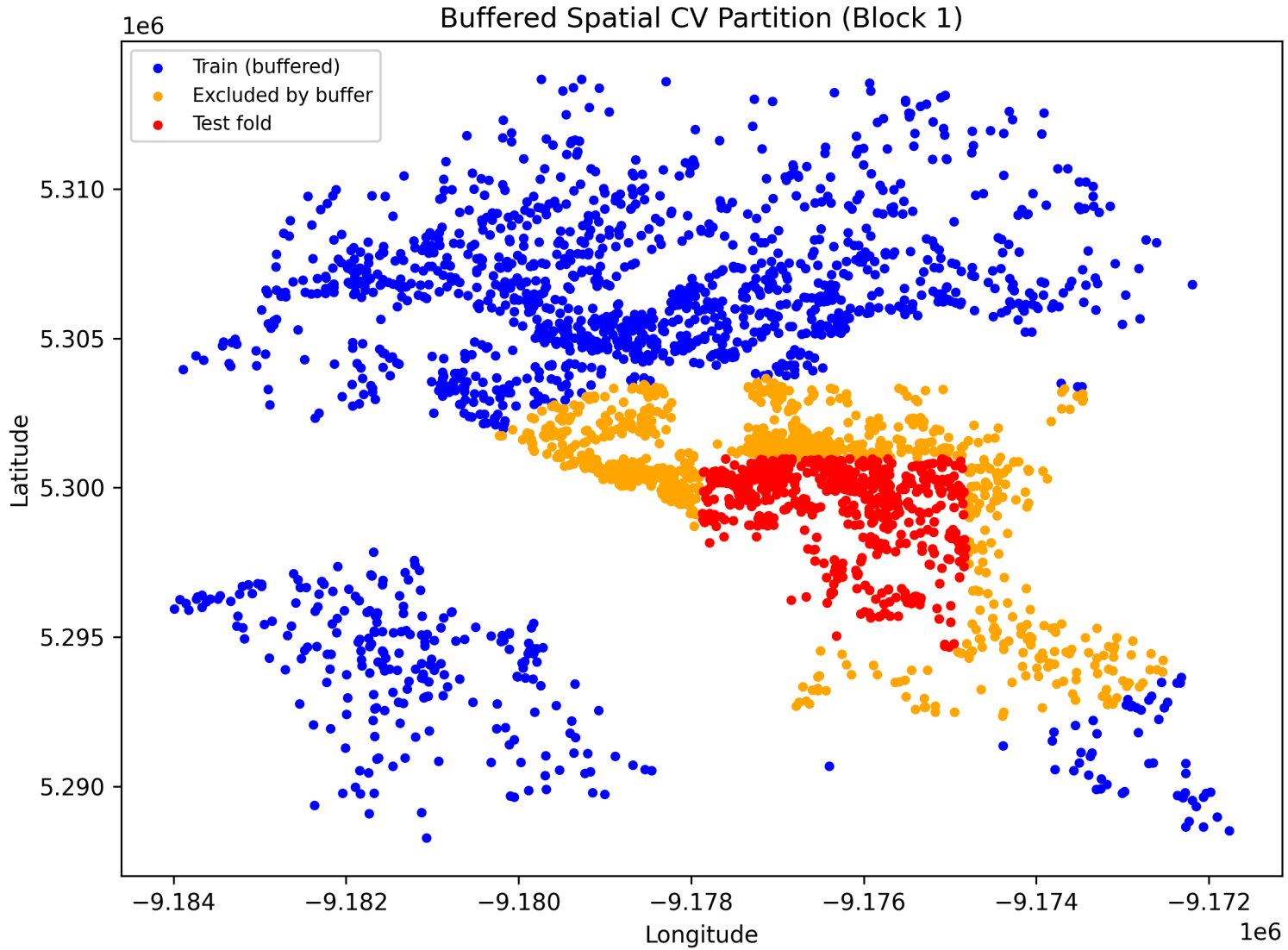
Buffered Spatial Cross-Validation

Concept:

- For each test location, exclude all training points within a **buffer** distance (e.g., based on spatial autocorrelation range).
- Ensures strict spatial separation between train and test sets.
- **Buffer size controls the minimum distance between test and training samples**.
- Especially useful for point data with spatial clustering or autocorrelation.

Buffer Size Selection:

- **Rule of thumb:** 2-3× correlation range
- **Empirical:** Test multiple buffer sizes
- **Theoretical:** Based on variogram range parameter


Buffered Spatial CV Partition (Block 1)
1e6
Train (buffered)
Excluded by buffer
Test fold
5.310
5.305
5.300
5.295
5.290
Latitude
−9.184
−9.182
−9.180
−9.178
−9.176
−9.174
−9.172
1e6
Longitude

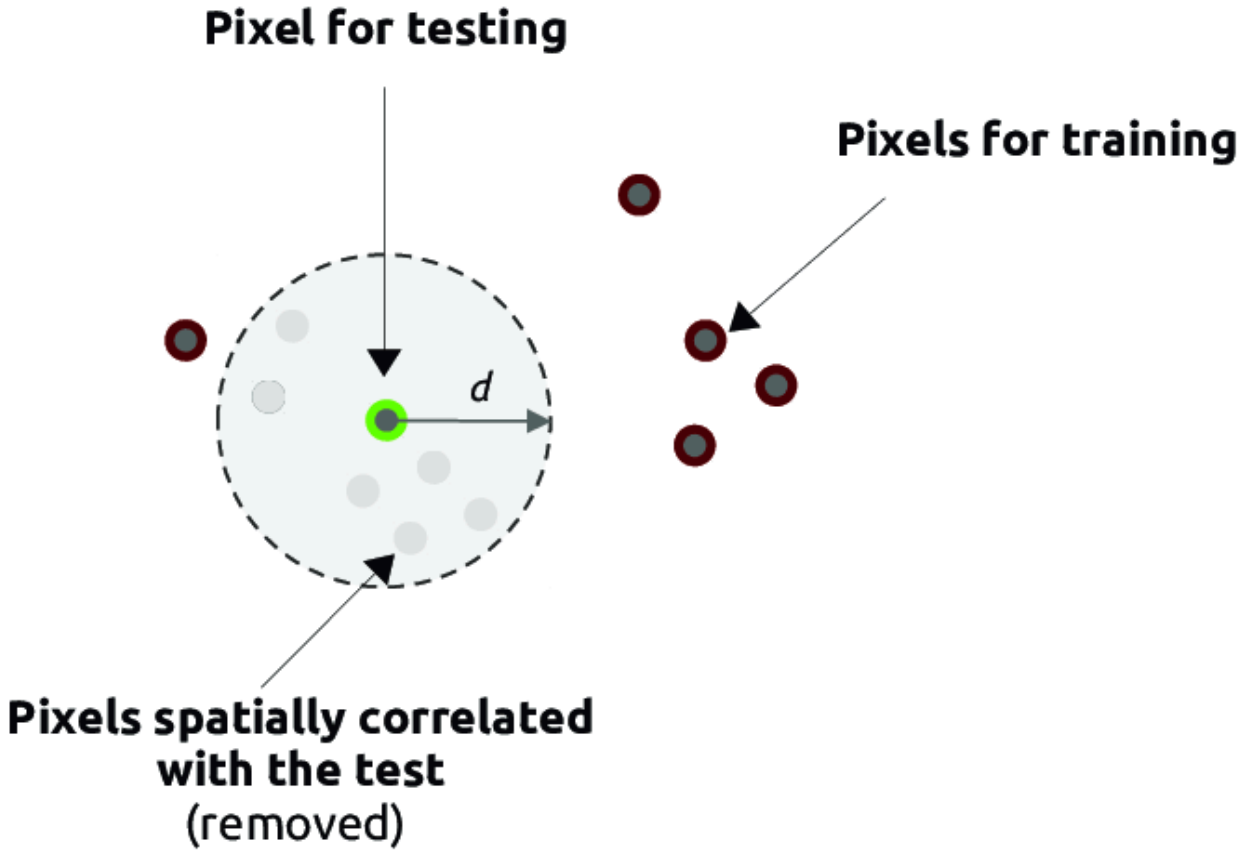
Leave-One-Out Spatial Cross-Validation (LOO-SCV)

Concept:

- For each observation, use it as the test set and all other observations as the training set.
- Strict independence: no overlap between train and test.

Spatial LOO Variant:

- Exclude not only the test point, but also **all points within a buffer distance** (see buffered CV).
- Useful for dense spatial samples and assessing model generalization at individual locations.



Leave-Location-Group-Out CV (LLGO)

Why LLGO?

- Standard Leave-One-Out (LOO) spatial CV tests model generalization at individual locations, but may not account for hierarchical or grouped spatial structure (e.g., cities, watersheds, farms).
- LLGO is designed for situations where data are naturally grouped by location, administrative unit, or other spatial hierarchy.
- Instead of leaving out single points, LLGO leaves out entire spatial groups, testing the model's ability to generalize to new regions or units.

How is LLGO different from LOO spatial CV?

- **LOO spatial CV:** Each test fold contains a single observation (or buffered area around it); evaluates point-level independence.
- **LLGO:** Each test fold contains all observations from a spatial group (e.g., all data from a city, watershed, or farm); evaluates group-level independence and transferability.
- LLGO is especially important when predictions will be made for new, unseen groups, not just new points within known groups.

Evaluation Metrics — what the scores mean

This deck keeps asking you to "report RMSE" and "Moran's I on residuals" — here are the exact definitions. The first three say *how big* the errors are; residual Moran's I says *whether what is left over is still spatially structured*.

Point-accuracy (per fold; residual $e_i = y_i - \hat{y}_i$)

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i e_i^2}$$

$$\mathrm{MAE} = \frac{1}{n}\sum_i |e_i|$$

$$R^2 = 1 - \frac{\sum_i e_i^2}{\sum_i (y_i - \bar{y})^2}$$

RMSE punishes large misses; MAE is robust; R^2 is the share of variance explained.

Residual spatial structure (weights w_{ij})

$$I = \frac{n}{S_0}\,\frac{\sum_i \sum_j w_{ij}\, e_i e_j}{\sum_i e_i^2}, \qquad S_0 = \sum_i \sum_j w_{ij}$$

$I \approx 0$ on residuals $\Rightarrow$ the model absorbed the spatial signal. $I > 0$ and significant $\Rightarrow$ leftover autocorrelation: folds are not independent and the model is missing spatial process.

Practical Recommendations Summary

For Areal Data:

- Use **contiguous block CV** or **leave-location-group-out**
- Buffer size: 1-2 neighboring units
- Report Moran's I on residuals

For Point Data:

- Use **buffered CV** with correlation range-based buffers
- Implement **target-oriented sampling** for clustered designs
- Check directional variograms

For Network Data:

- Use **leave-subgraph-out** CV
- Consider **network distance** in buffer calculations
- Evaluate **connectivity-based** accuracy metrics

Key Takeaways:

- **Use random CV** for spatial data only when sample is random
- **Always check residual spatial structure**

How to present results (recommended)

- Map residuals and local RMSE side-by-side with histogram / boxplot of residuals.
- Report Moran's I with p-value and a short interpretation sentence.
- Show empirical variogram with fitted model; annotate the range $\rightarrow$ explain chosen buffer/block size.
- Report distribution (mean, median, SD, IQR) of the metric across folds (not only the mean).
- Show PICP and PINAW overall and by region (table + map).
    - PICP (Prediction Interval Coverage Probability): fraction of observations whose true value falls inside the predicted interval.

$$\mathrm{PICP} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{y_i \in [\hat{y}_i^{\mathrm{lower}},\, \hat{y}_i^{\mathrm{upper}}]\}$$

Interpretation: PICP close to the nominal coverage (e.g., 0.95) indicates well-calibrated intervals.

- PINAW (Prediction Interval Normalized Average Width): average interval width normalized by the data range (smaller is tighter).

$$\mathrm{PINAW} = \frac{1}{n(y_{\max} - y_{\min})} \sum_{i=1}^{n} \left(\hat{y}_i^{\mathrm{upper}} - \hat{y}_i^{\mathrm{lower}}\right)$$

  Interpretation: trade-off between interval width and coverage — report both PICP and PINAW together.

How to make them spatial

- By-region (administrative units / strata)
  - Compute PICP and PINAW per region; show a table + choropleth map.
- Per-fold (CV fold) or per-cluster
  - Report PICP/PINAW distributions across folds/clusters to check transferability.
- Local / moving-window
  - Compute PICP/PINAW in a moving window or k-nearest neighborhood to map local calibration / sharpness.
- Map coverage flags (covered = 0/1) and test spatial autocorrelation (Moran's I) of coverage failures.
- Inspect PINAW spatially (map interval widths) to see where uncertainty is large.

Practical tips

- Always show both PICP (calibration) and PINAW (sharpness). High PICP + huge PINAW is not useful; low PINAW + poor PICP is overconfident.
- For region-level PICP, include confidence intervals (binomial) so you know whether deviations from nominal are significant.
- If coverage failures cluster spatially, investigate covariates / missing processes and consider location-dependent uncertainty models (heteroskedastic models, spatial random effects, spatial conformal methods).

Worked example — random CV is optimistic (Python · meuse)

We predict **log(zinc)** from distance-to-river (**dist**) and elevation (**elev**), then score *one* linear model two ways: scattered random folds vs. whole 500 m spatial blocks held out together. Only the split changes.

```
import numpy as np, pandas as pd, verde as vd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

df = pd.read_csv("data/meuse/meuse.csv")
df["log_zinc"] = np.log(df["zinc"])
X = df[["dist", "elev"]].to_numpy()
y = df["log_zinc"].to_numpy()
coords = df[["x", "y"]].to_numpy()              # (n, 2): easting, northing

def cv_report(splits):  # mean + R^2 spread over folds
    rmse, mae, r2 = [], [], []
    for tr, te in splits:
        m = LinearRegression().fit(X[tr], y[tr]); p = m.predict(X[te])
        rmse.append(mean_squared_error(y[te], p) ** 0.5)
        mae.append(mean_absolute_error(y[te], p)); r2.append(r2_score(y[te], p))
    return np.mean(rmse), np.mean(mae), np.mean(r2), np.std(r2)

random_cv = KFold(n_splits=5, shuffle=True, random_state=0)  # optimistic
block_cv = vd.BlockKFold(spacing=500, n_splits=5,
                         shuffle=True, random_state=0)  # honest
block_ss = vd.BlockShuffleSplit(spacing=500, n_splits=30,
                                test_size=0.3, random_state=0)
print("random  ", cv_report(random_cv.split(X)))
print("block", cv_report(block_cv.split(coords)))  # verde on coords
print("block-SS", cv_report(block_ss.split(coords)))
```

Worked example — the honest-vs-optimistic gap

Holding out whole blocks barely raises the average error, yet mean R^2 collapses and the fold-to-fold spread explodes — the random-CV number hid how unevenly the model transfers (meuse, n = 155).

scheme	held out	RMSE	MAE	mean R^2	R^2 sd
random 5-fold (`KFold`)	scattered points	0.43	0.34	**0.62**	0.13
`verde.BlockKFold` (500 m)	whole blocks	0.45	0.36	**0.40**	0.35
`verde.BlockShuffleSplit`	random block sets	0.45	0.36	0.58	0.09
`GroupKFold` (fallback)	whole blocks	0.45	0.36	0.55	0.06

```
# Fallback if verde missing: grid block ids, leave blocks out.
from sklearn.model_selection import GroupKFold
step = 500.0
bx = np.floor((df.x - df.x.min()) / step).astype(int)
by = np.floor((df.y - df.y.min()) / step).astype(int)
block_id = bx.astype(str) + "_" + by.astype(str)
print("group   ", cv_report(GroupKFold(n_splits=5).split(X, y, block_id)))
```

Caveat — is spatial CV the right yardstick for map accuracy?

Distance-aware CV is now standard practice, but its role for *map-accuracy assessment* is genuinely debated — the honest answer depends on how your data were sampled and what you want to estimate.

- **Probability sampling:** design-based validation on a held-out probability sample is unbiased, and spatial CV can be *pessimistic* — Wadoux, Heuvelink, de Bruin & Brus (2021), *Ecological Modelling* 457:109692.
- **Convenience / clustered sampling:** random CV is over-optimistic (as shown above), so distance-aware schemes are warranted.
- **Same-topic refinements:** NNDM LOO matches the train–test nearest-neighbour-distance distribution to the prediction domain (Milà et al. 2022); kNNDM extends it to k-fold (Linnenbrink et al. 2024) — both surface as **cv_nndm** in blockCV.

Takeaway: choose the validation design from your *sampling design* and *inference goal* — not by default.

blockCV (R) — field-standard reference

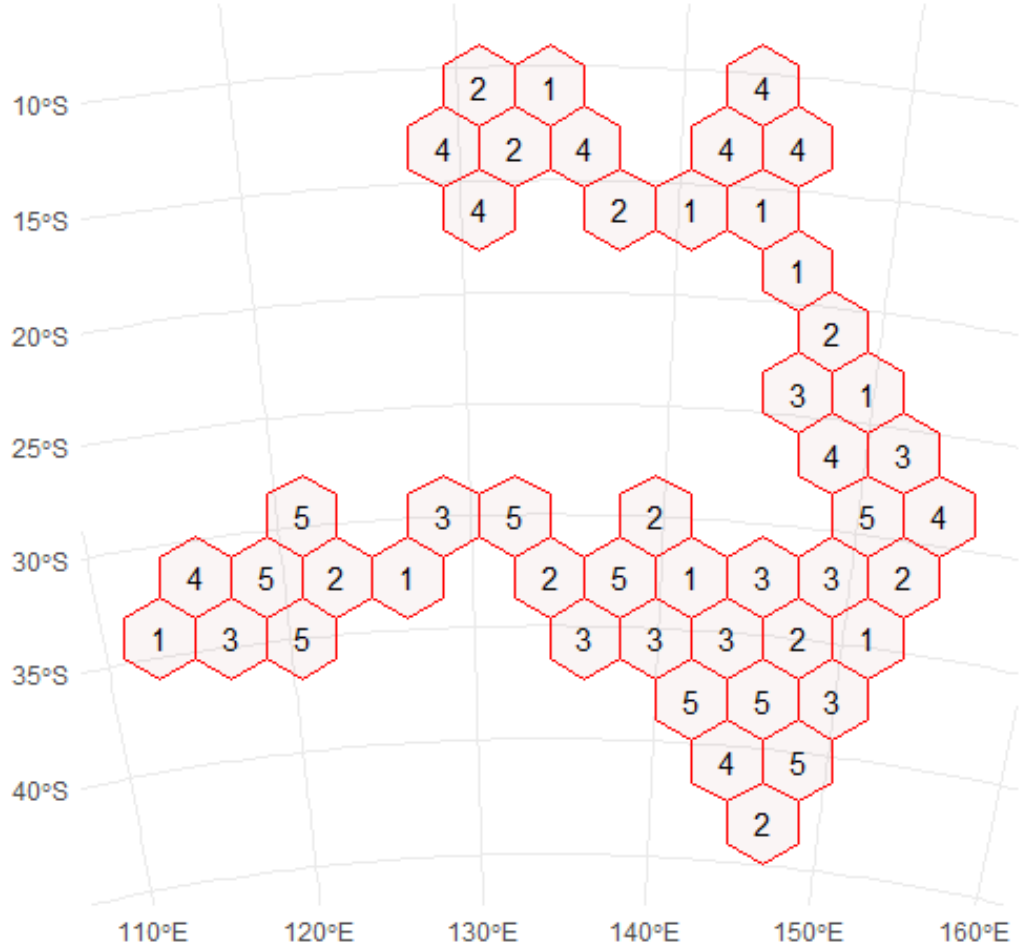
The Python recipe above mirrors standard R practice: **blockCV** (v3) is the de-facto toolkit for spatial CV in species-distribution modelling. We keep its walkthrough as a cross-reference — its method names map one-to-one onto the schemes we just covered (**cv_spatial** = blocks, **cv_cluster** = clustering, **cv_buffer** = buffered LOO, **cv_nndm** = NNDM).

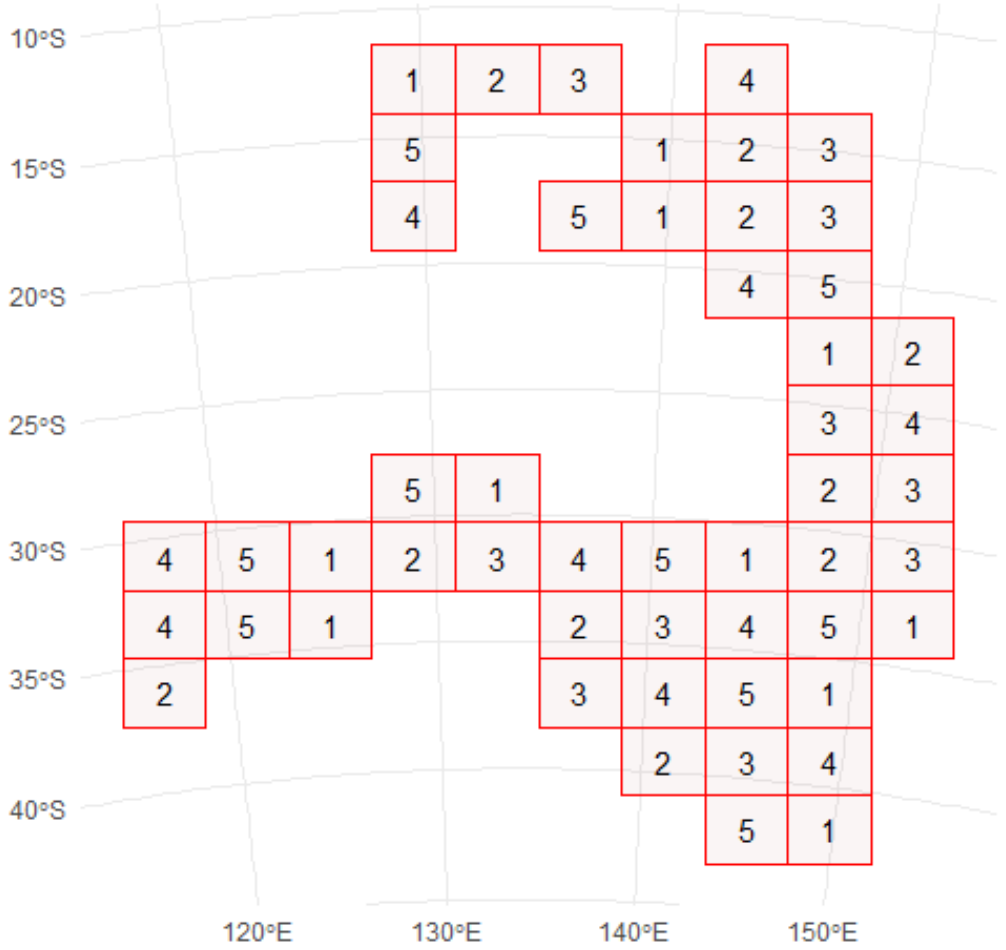
1. Why spatial cross-validation (SCV)?
2. The blockCV toolbox at a glance
3. Methods
    - Spatial blocks (**cv_spatial**)
    - Spatial/Environmental clustering (**cv_cluster**)
    - Buffered spatial LOO (**cv_buffer**)
    - NNDM LOO (**cv_nndm**)
4. Choosing a block/buffer size (**cv_spatial_autocor**, **cv_block_size**)
5. Plotting & diagnostics (**cv_plot**, **cv_similarity**)
6. Practical tips & references

Method 1 — Spatial blocks (`cv_spatial`)

Idea. Cover the region with blocks (hex or square), then assign entire blocks to folds so train/test are spatially separated.
When to use. Classic k-fold SCV for gridded or regional studies; fair when blocks approximate the spatial scale of dependence.
Step-by-step:

1. **Choose block size** (metres) or grid (`rows_cols`).
2. **Make blocks**: hex (default) or squares.
3. **Assign folds** (k): random / systematic / checkerboard.
4. **Check balance** of train/test counts (tune `iteration` for better balance).
5. **Plot** with `cv_plot` to verify spatial separation.


10°S
15°S
20°S
25°S
30°S
35°S
40°S
110°E
120°E
130°E
140°E
150°E
160°E


10°S
15°S
20°S
25°S
30°S
35°S
40°S
120°E
130°E
140°E
150°E

`cv_spatial`: Tips & pitfalls

- **Scale matters**: set block `size` near the spatial autocorrelation range of your predictors or residuals.
- Use **random** with `iteration` to improve class balance in binary outcomes.
- If points cluster strongly, **systematic** or **checkerboard** can avoid empty test folds.
- Don’t mix coordinate units — `size` is interpreted in **metres**.

Method 2 — Spatial / Environmental clustering (`cv_cluster`)

Idea. Cluster points in **geographic** space (coords) or **environmental** space (rasters). Assign clusters to folds.

When to use.

- Spatial k-means: flexible shapes vs. rigid blocks.
- Environmental clustering: test extrapolation to novel environments, or de-bias by environment.

What you can cluster on

Coordinates of points (spatial clustering)

- If you do not supply raster covariates (r = NULL), then clustering is done in “spatial space”, using the coordinates of your x sample points.
- Basically, cv_cluster(..., x = pa_data, k = something) clusters the x,y coordinates by k-means.

Environmental variables (covariates)

- If you supply a raster stack / SpatRaster object via r, then clustering is done in the space of environmental covariates.
- You can choose whether clustering is done using just the environmental values at the sample points, or across all raster cells (“raster space”) – depending on the argument raster_cluster.

Method 3 — Buffered spatial LOO (`cv_buffer`)

Idea. For each target point, **buffer** by a distance and exclude all points in that buffer from training. Test on the target (optionally more).
When to use. Strict independence at a chosen distance; robust for dense or irregular samples; good for SDMs and map validation.
Step-by-step:

1. Pick buffer **`size`** (metres), ideally near the **autocorrelation range**.
2. For each record *i*:
    - **Test set** = record *i*.
    - **Train set** = all records **outside** the buffer around *i*.
3. Repeat for all records → **LOO-like** folds (often many!).
4. Plot a subset of folds to inspect.

Method 4 — NNDM LOO (`cv_nndm`)

Idea. A fast C++ LOO that matches the **nearest-neighbour distance distribution** in train vs. test to mimic prediction settings (target domain).
When to use. When simple buffers don't reflect target sampling density or distribution; when you want **distribution-aware** separation.

Step-by-step:

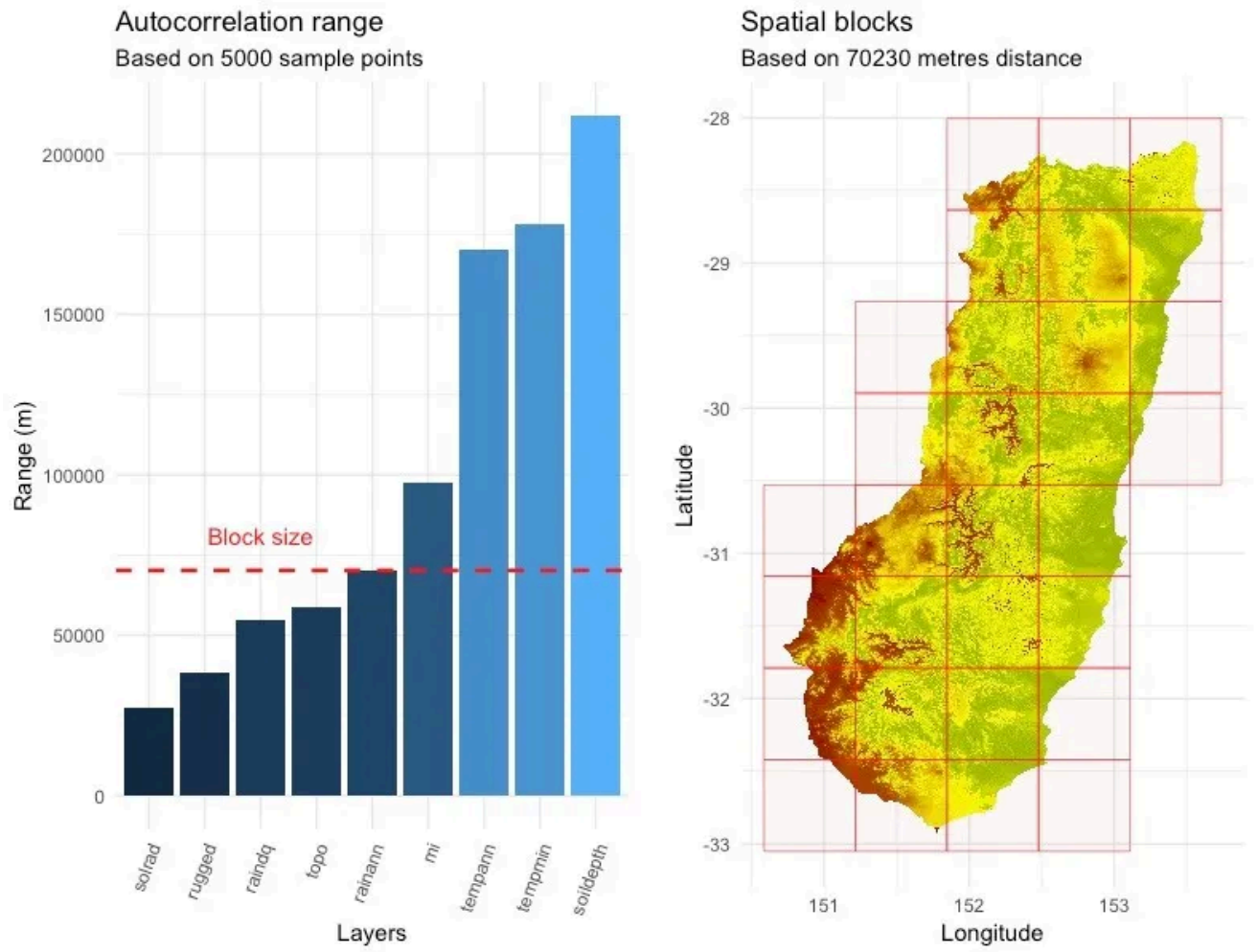
1. Provide sample `x` (and `column` for binary data).
2. Provide a **raster** or spatial extent representing the **prediction domain** (`r`).
3. Choose `size` (distance scale), `num_sample` (sampling points from domain), `sampling` ("regular"/"random"), and a minimum train fraction `min_train`.
4. Run and **plot** selected folds; evaluate.

Blocks vs. Clusters vs. Buffers vs. NNDM — quick contrasts

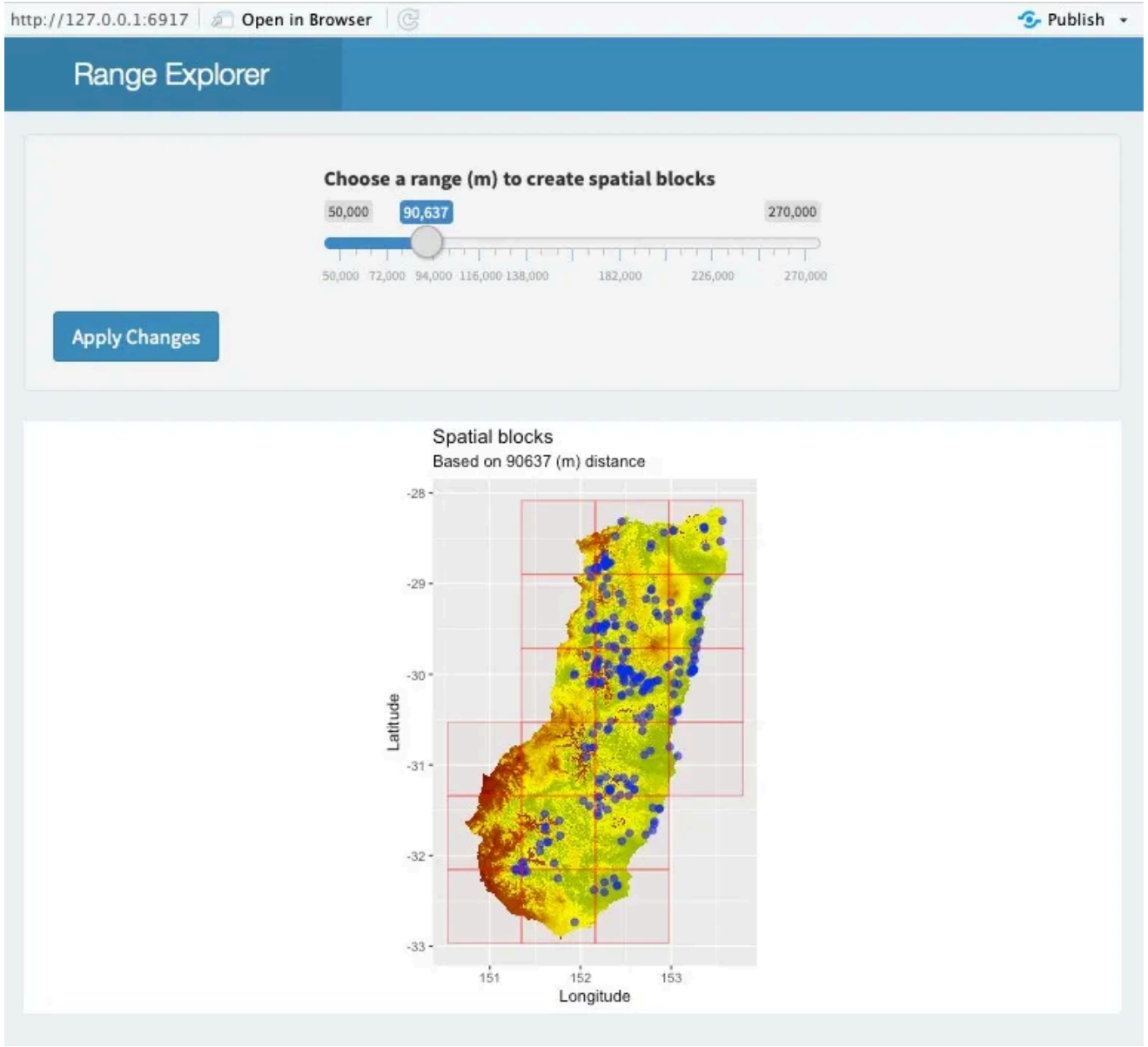
- **Blocks**: simple geometry; k folds; controlled **spatial scale** via `size`.
- **Clusters**: data-driven shapes; can be environmental; k folds; **balance** varies with `k`.
- **Buffers (LOO)**: strict separation at distance; **many** folds; heavy compute but principled.
- **NNDM (LOO)**: distance-distribution matching to target domain; **data- and domain-aware**.

Choosing a distance: `cv_spatial_autocor`

Goal. Estimate the **effective range of spatial autocorrelation** to inform block/buffer size.



Interactive block-size chooser: `cv_block_size`



Checking environmental similarity: `cv_similarity`

Why. Environmental novelty in test folds can bias evaluation.
What. Compute **MESS** or **L1/L2** distances between train and test environments.

```
cv_similarity(cv = ecv, x = pa_data, r = rasters, method = "MESS")
```

Negative MESS suggests extrapolation — report it and consider adjusting folds.

Practical setup & evaluation checklist

- **Project CRS** defined; distances in **metres**.
- **Pick a method** (blocks / clusters / buffer / NNDM) aligned to your task and domain.
- **Choose size** via autocorrelation or domain knowledge.
- **Balance**: tune **k** and **iteration** (blocks), or **k** (clusters).
- **Avoid leakage**: standardise features **within each train fold** only.
- **Aggregate metrics** across folds; report distributions, not just means.
- **Visualise & document**: include fold maps in your reports.

References

- **blockCV tutorials** (vignettes): introduction & SDM examples.
  - Tutorial 1: *How to create block cross-validation folds*.
  - Tutorial 2: *Block cross-validation for species distribution modelling*.
- Valavi R., Elith J., Lahoz-Monfort J., Guillera-Arroita G. (2019). *Methods in Ecology & Evolution* 10:225–232.
- Figures used on selected slides come from blockCV docs (README/tutorials) and the *Methods Blog* post introducing blockCV.